

Computer Organization & Architecture

Unit- 6 Central Processing Unit (Part - 2)

Prepared By: Sweetu D. Sureja
Information Technology Department
V.V.P. Engineering College, Rajkot.

- Introduction
- General Register Organization
- Stack Organization
- Instruction format
- Addressing Modes
- **Data transfer and manipulation**
- **Program control**
- **Reduced Instruction Set Computer (RISC) & Complex Instruction Set Computer (CISC)**

Data Transfer and Manipulation

- ♦ Most computer instructions can be classified into three categories:

1. Data transfer instructions 2. Data manipulation instructions 3. Program control instructions

- ♦ **Data transfer instructions:**

- ♦ It cause transfer of data from one location to another without changing the binary information content.
- ♦ The transfers are between memory and processor registers, between processor registers and input or output, and between the processor registers themselves.
- ♦ The **load** instruction has been used mostly to designate a transfer from memory to a processor register, usually an accumulator.
- ♦ The **store** instruction designates a transfer from a processor register into memory.
- ♦ The **move** instruction has been used in computers with multiple CPU registers to designate a transfer from one register to another or data transfers between CPU registers and memory or between two memory words.

- ♦ The **exchange** instruction swaps information between two registers or a register and a memory word.
- ♦ The **input and output** instructions transfer data among processor registers and input or output terminals.
- ♦ The **push and pop** instructions transfer data between processor registers and a memory stack.

Name	Mnemonic
Load	LD
Store	ST
Move	MOV
Exchange	XCH
Input	IN
Output	OUT
Push	PUSH
Pop	POP

- ♦ **Data Manipulation Instructions:** It perform operations on data and provide the computational capabilities for the computer. The data manipulation instructions in a typical computer are usually divided into three basic types:

1. Arithmetic instructions
2. Logical and bit manipulation instructions
3. Shift instructions

1) Arithmetic Instructions

- ♦ The four basic arithmetic operations are addition, subtraction, multiplication, and division.
- ♦ **Increment instruction** adds 1 to the value stored in a register or memory word. **Decrement instruction** subtracts 1 from a value stored in a register or memory word.
- ♦ **Add, Subtract, Multiply, Divide** instructions may be available for different types of data.
- ♦ Instruction **add with carry** performs the addition on two operands
Instruction **subtract with borrow** subtracts two words
- ♦ **Negate instruction** forms the 2's complement of a number.

Name	Mnemonic
Increment	INC
Decrement	DEC
Add	ADD
Subtract	SUB
Multiply	MUL
Divide	DIV
Add with carry	ADDC
Subtract with borrow	SUBB
Negate (2's complement)	NEG

Logical & Bit Manipulation Instructions

Name	Mnemonic
Clear	CLR
Complement	COM
AND	AND
OR	OR
Exclusive-OR	XOR
Clear carry	CLRC
Set carry	SETC
Complement carry	COMC
Enable interrupt	EI
Disable interrupt	DI

Shift Instructions

Name	Mnemonic
Logical shift right	SHR
Logical shift left	SHL
Arithmetic shift right	SHRA
Arithmetic shift left	SHLA
Rotate right	ROR
Rotate left	ROL
Rotate right through carry	RORC
Rotate left through carry	ROLC

Program Control

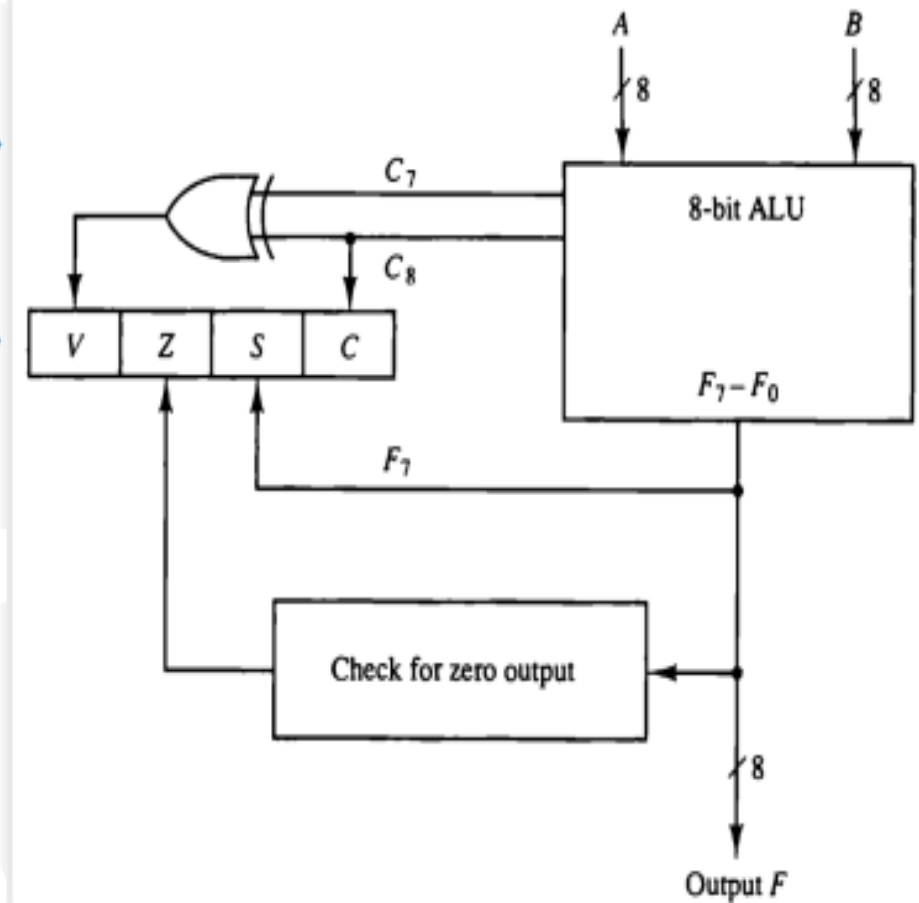
- Program control instructions **change the normal flow of execution** of a program.
- Normally, instructions run one after another (sequentially).
- But these instructions **modify the Program Counter (PC)**, so execution jumps somewhere else.

In short:

They control “what instruction runs next.”

Name	Mnemonic
Branch	BUN
Jump	JMP
Skip	SKP
Call	CALL
Return	RET
Compare (by subtraction)	CMP
Test (by <u>ANDing</u>)	TST

- ◆ Status bits are also called condition-code bits or flag bits.
- ◆ The four status bits are symbolized by C, S, Z, and V.
- ◆ Bit C (carry) is set to 1 if the end carry C_8 is 1. It is cleared to 0 if the carry is 0.
- ◆ Bit S (sign) is set to 1 if the highest-order bit F_7 is 1. It is set to 0 if the bit is 0.
- ◆ Bit Z (zero) is set to 1 if the output of the ALU contains all 0's.
In other words, $Z = 1$ if the output is zero and $Z = 0$ if the output is not zero.
- ◆ Bit V (overflow) is set to 1 if the exclusive-OR of the last two carries is equal to 1



Example

- Status Bit Conditions in 8-Bit ALU

Given:

A = **11110000**

B = **00010100**

Operation:

- Perform **A – B** using 8-bit ALU
- $A - B = A + (2\text{'s complement of } B)$

Step 1: 2's Complement of B

- B = 00010100
- 1's complement ($\bar{B}$) = 11101011
- $\bar{B} + 1 = \mathbf{11101100}$

Step 2: Addition

A = 11110000
+ ($\bar{B} + 1$) = 11101100

11011100

Carry Out (C8) = 1

Output:

•F = **11011100**

Status Bits:

Carry (C) = **1**

Sign (S) = **1**

Zero (Z) = **0**

Overflow (V) = **0**

Carry (C) = 1

→ Carry is generated from the MSB (C8). Since addition produced a carry out, **C = 1**.

Sign (S) = 1

→ Sign bit is the MSB of result (F). Here F = **11011100**, MSB = 1

→ **negative number**, so **S = 1**.

Zero (Z) = 0

→ Result is not zero ($F \neq 00000000$), so **Z = 0**.

Overflow (V) = 0

→ Overflow = $C7 \oplus C8$

→ Here $C7 = 1$ and $C8 = 1 \rightarrow 1 \oplus 1 = 0 \rightarrow$ **no overflow**, so **V = 0**.

- ♦ Program interrupt refers to the transfer of program control from a currently running program to another service program as a result of an external or internal generated request.
- ♦ **Types of Interrupts:**
 - 1) **External interrupts:**
 - ♦ External interrupt arise due to,
 - ♦ Input-output (I/O) devices
 - ♦ Timing device
 - ♦ Circuit monitoring the power supply
 - ♦ Any other external source.
 - ♦ Examples that cause external interrupts are
 - ♦ I/O device requesting transfer of data
 - ♦ I/O device finished transfer of data
 - ♦ Power failure.
 - ♦ External interrupts are asynchronous.
 - ♦ It depends on external conditions that are independent of the program being executed at the time.

2) Internal interrupts (Traps):

- ◆ Internal interrupts arise from illegal or erroneous use of an instruction or data. Internal interrupts are also called traps.
- ◆ Examples of interrupts caused by internal error conditions like
 - ◆ Register overflow
 - ◆ Attempt to divide by zero
 - ◆ An invalid operation code
 - ◆ Stack overflow
- ◆ Internal interrupts are synchronous with the program. If the program is rerun, the internal interrupts will occur in the same place each time.

Reduced Instruction Set Computer (RISC)

- ♦ A number of computer designers recommended that computers use fewer instructions with simple constructs so they can be executed much faster within the CPU without having to use memory
- ♦ The concept of RISC architecture involves an attempt to reduce execution time by simplifying the instruction set of the computer. The major characteristics of a RISC processor are:
 - ♦ Relatively few instructions
 - ♦ Relatively few addressing modes
 - ♦ Memory access limited to load and store instructions
 - ♦ All operations done within the registers of the CPU
 - ♦ Fixed-length, easily decoded instruction format
 - ♦ Single-cycle instruction execution
 - ♦ Hardwired rather than microprogrammed control
 - ♦ A relatively larger number of registers in the processor unit.
- ♦ Efficient instruction pipeline
- ♦ Compiler support for efficient translation of high level programs into machine language programs.

Complex Instruction Set Computer (CISC)

- ♦ The design of an instruction set for a computer must take into consideration not only machine language constructs, but also the requirements imposed on the use of high-level programming languages.
- ♦ The major characteristics of CISC architecture are:
 - ♦ A large number of instructions-typically from 100 to 250 instructions
 - ♦ Some instructions that perform specialized tasks and are used infrequently
 - ♦ A large variety of addressing modes-typically from 5 to 20 different modes
 - ♦ Variable-length instruction formats
 - ♦ Instructions that manipulate operands in memory

State the differences between RISC and CISC.

It is a Reduced Instruction Set Computer.	It is a Complex Instruction Set Computer.
It based on software to optimize the instruction set.	It based on hardware to optimize the instruction set.
It is a hard wired unit of programming in the RISC Processor.	Microprogramming unit in CISC Processor.
It requires multiple register sets to store the instruction.	It requires a single register set to store the instruction.
RISC has simple decoding of instruction.	CISC has complex decoding of instruction.
Uses of the pipeline are simple in RISC.	Uses of the pipeline are difficult in CISC.
It uses a limited number of instruction that requires less time to execute the instructions.	It uses a large number of instruction that requires more time to execute the instructions.
The execution time of RISC is very short.	The execution time of CISC is longer.
RISC architecture can be used with high-end applications like telecommunication, image processing, video processing, etc.	CISC architecture can be used with low-end applications like home automation, security system, etc.
It has fixed format instruction.	It has variable format instruction.
The program written for RISC architecture needs to take more space in memory.	Program written for CISC architecture tends to take less space in memory.
Example of RISC: ARM, PA-RISC, Power Architecture, Alpha, AVR, ARC and the SPARC.	Examples of CISC: VAX, Motorola 68000 family, System/360, AMD and the Intel x86 CPUs.

QUESTIONS ASKED IN GTU EXAM

Q.1 Enlist various kinds of addressing modes. Explain any five of same and support your answer by taking small example.

Q.2 Write two address, one address and zero address instructions program for the following arithmetic expression: $X = (A + B) * (C - D / E) + F * G$

Q.3 Explain status bit conditions with neat diagram.

Q.4 State the differences between RISC and CISC.

Q.5 Summarize following addressing modes with example. 1) Implied mode 2) Register mode

Q.6 Criticize Three-Address Instructions and Zero address instruction with common example.

Q.7 Differentiate RISC and CISC architecture.

Q.8 What is difference between two address and three address instructions?

Q.9 What is difference between direct and indirect addressing mode?

Q.10 What addressing mode means? Explain any three addressing modes in detail with example.

Q.11 Explain register stack and memory stack.

Q.12 Write a program to evaluate the arithmetic statement: $A*B+C*D+E$ i. Using a stack organized computer.

Q.13 List down six major characteristics of RISC processors.

Q.14 Enlist different status bit conditions.

Q.15 What is addressing mode? Explain direct and indirect addressing mode with example.

Q.16 Compare and contrast RISC and CISC.